Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

INF205

Ressurseffektiv programmering

2 Minnehandtering

2.1 Kallstakken og haugen

2.2 Vanlege minnehandteringsfeil

2.3 Containerklasser (kopiering og flytting)

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

Problemliste B

INF205

20. februar 2026

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

2 Minnehandtering

2.1 Kallstakken og haugen

2.2 Vanlege feil

2.3 Containerklasser

Ordliste

container	container, konteinar
copy assignment operator	kopieringsoperator
deep copy	djupnekopi [dybdekopi]
moving	flytting
rule of three/five	trear-/femmarregelen
shallow copy	overflatekopi

Common mistakes

Repetition: We talked about three main kinds of mistakes in manual memory management. What were they? (1) … … (2) … … (3) … …

Strategies that we had identified for trying to avoid these mistakes:

1) **Avoid manual memory management** if possible; use the stack, not the heap.
   - Don't work with pointers unless there is a clear advantage. Don't pass by reference without a good reason.

2) **Assign clear responsibilities** for what part of the code is to allocate each data item that you create on the heap.
   - Create a "container" that "owns" it. If suitable, library (*e.g.* STL) containers.

Smart pointers are very elementary containers. They have ownership over the object to which they point.

Instead of T*, where T is the type, we can use std::unique_pointer<T> or std::shared_pointer<T>.

See Core Guidelines I.11.

Constructors and destructors

Constructor: A method that is called when an object is **allocated**.
Destructor: A method that is (implicitly) called when an object is **deallocated**.

They are not mandatory (as we have seen); use them if you need to specify some functionality for this purpose. Most typically:

- Provide a **constructor** if you want to give the user control over how the private properties of an object are initialized.
- There are also special "copy constructors" and "move constructors". (We will discuss them in detail at a later stage in the course.)
- Provide a **destructor** if your memory management strategy requires it; there might be properties stored as pointers that need to be deleted.

```
class BookIndex {
public:
  BookIndex(int c, int s, int p);
  ~BookIndex();
  …
};
```

```
BookIndex::BookIndex(int c, int s, int p) {
  this->chapter = c; this->section = s; this->page = p;
}
BookIndex::~BookIndex() {
  cout << "Deleting a BookIndex object.\n";
}
```

Constructors and destructors

General rule: For every "new" there must be a matching "delete".

The destructor **T::~T()** is called when an object of type T is deallocated.

This is the case both for objects on the stack and on the heap:

```
void function_name(…)
{
  // constructor is called
  T tobject;
  …
  // destructor is called
  return;
}
```

```
{
  …
  // constructor is called
  T* tpointer = new T;
  …
  // destructor is called
  delete T;
}
```

```
class T
{
public:
  …
  ~T() { delete this->p; }
  …
private:
  S* p …
}
```

If T has ownership over p, this must be done! Without it, there would be a **memory leak**!

There might by other properties that do *not* need to be deallocated manually. (*Why?*)

Constructors and destructors

Core Guidelines:

- C.20: "If you can avoid defining default operations, do."
  - "This is known as "the **rule of zero**." Define zero constructors or destructors if it can be done without creating an inconsistent state.
  - A simple and good reason for defining a constructor is to force the user to provide some information that is required.
  - Define a constructor in cases where it does not make sense to initialize the object's properties to some specified default value.

- **C.30**: "Define a destructor if a class needs an explicit action at object destruction." And related, **C.31**: "All resources acquired by a class must be released by the class's destructor."
  - If a data structure needs to be built up (memory allocated, *etc.*), this normally requires both a constructor and a destructor.
  - For such cases, we will learn the *rule of three* and the *rule of five*.

Rule of three

Container objects take **ownership**, *i.e.*, lifetime and deallocation responsibility. The programmer needs to take care of this whenever there are data subject to **manual memory management** (*new* and *delete*) in a **self-designed container**.

The programmer needs to take care of this whenever there are data subject to **manual memory management** (*new* and *delete*) in a **self-designed container**.

"**Rule of three:**" For a container, implement at least

(1) destruktør,
(2) kopieringskonstruktør,
(3) kopieringsoperator.

(1) destructor,
(2) copy constructor,
(3) copy assignment operator.

Most often you will then also need to implement **(0)** a constructor.

At least implement **(1) the destructor**!
If **(2)** and **(3)** are not there, **forbid copying**.

Rule of three: (1) Destructor

Container objects take **ownership**, *i.e.*, lifetime and deallocation responsibility. The programmer needs to take care of this whenever there are data subject to **manual memory management** (*new* and *delete*) in a **self-designed container**.

Example: Let us assume that **class T** has one property for which it has ownership, a pointer p to class S that points to an array of 1000 S elements.

It is typical for the owned content, if manual memory management needs to be done, to be allocated in the constructor, **T::T()** and/or **T::T(…)**.

```
T tobject;
```

```
T* tpointer = new T;
```

```
class T
{
public:
  T() { this->p = new S[1000]; }
  ~T() { delete[] this->p; }
  …
private:
  S* p = nullptr;
  …
}
```

Rule of three: (2) Copy constructor

The **copy constructor T::T(const T&** orig**)** is called when the following two are done at the same time: **(1) allocation** of an object, so that a constructor needs to be called, and its **(2) initialization** to the value of a pre-existing object that continues to exist.

Examples for when the **copy constructor** is called:

```
// default constructor
T tfirst;
…
// copy constructor
T tsecond = tfirst;
```

```
void func(T param) { … }

int main() {
  T tobject;
  …
  // copy constructor
  func(tobject);
}
```

after running the copy constructor, the same content must exist in memory twice!

```
class T
{
public:
  T() { this->p = new S[1000](); }
  T(const T& original) {
    this->p = new S[1000]();
    std::copy(
      original.p, original.p+1000,
      this->p
    );
  }
  …
}
```

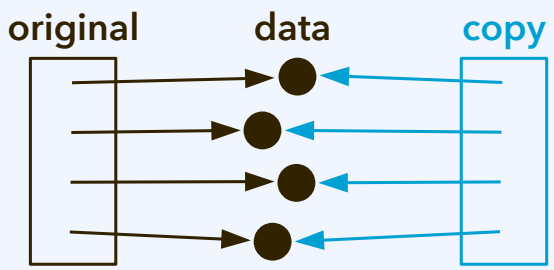
std::copy can be used for data that are contiguous in memory

1. Create space for the duplicate.
2. Now write the duplicate into it.

Copying an object

Shallow copy:

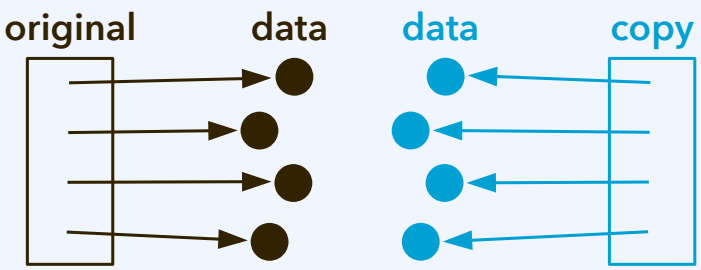
Standard copying, such as if there is no handwritten copy constructor or copy assignment operator, will simply **copy the value of pointers**, *not the content* to which they point.



After shallow copying, the content will exist **once in memory**. This can be appropriate when the content is **not owned** but just pointed at.

Deep copy:

Standard copying, such as if there is no handwritten copy constructor or copy assignment operator, will simply **copy the value of pointers**, *not the content* to which they point.



After deep copying, content exists **twice in memory**. Design following the concept of a "container" that **uniquely "owns"** its content requires deep copying.

Rule of three: (3) Copy assignment operator

The **copy assignment operator** technically is an overloaded "=" operator:

```
T& T::operator=(const T& rhs) { … }
```

Difference from the copy constructor:

- Object already exists, hence *no initial allocation* of memory for content.
- But *deallocate pre-existing content*.

```
// default constructor
T tfirst, tsecond;
…
// copy assignment
tsecond = tfirst;
```

A **copy assignment** is done whenever we copy the value of one variable to another, **both existed** before, and **both continue to exist**.

```
class T
{
public:
  T() { this->p = new S[1000](); }
  T& operator=(const T& rhs) {
    if(&rhs == this) return *this;
    delete this->p;
    this->p = new S[1000];
    std::copy(
      rhs.p, rhs.p+1000, this->p
    );
    return *this;
  }
  …
}
```

Note that a reference to *this needs to be returned.

Copy assignment operator

The **copy assignment operator** technically is an overloaded "=" operator:

```
T& T::operator=(const T& rhs) { … }
```

Difference from the copy constructor:

- Object already exists, hence *no initial allocation* of memory for content.
- But *deallocate pre-existing content if necessary*.

```
// default constructor
T tfirst, tsecond;
…
// copy assignment
tsecond = tfirst;
```

A **copy assignment** is done whenever we copy the value of one variable to another, **both existed** before, and **both continue to exist**.

after running the copy assignment, the same content must exist in memory twice!

```
class T
{
public:
  T() { this->p = new S[1000](); }
  T& operator=(const T& rhs) {
    if(&rhs == this) return *this;

    std::copy(
      rhs.p, rhs.p+1000, this->p
    );
    return *this;
  }
  …
}
```

Rule of three and rule of five

Container objects take **ownership**, *i.e.*, lifetime and deallocation responsibility. The programmer needs to take care of this whenever there are data subject to **manual memory management** (*new* and *delete*) in a **self-designed container**.

The programmer needs to take care of this whenever there are data subject to **manual memory management** (*new* and *delete*) in a **self-designed container**.

"**Rule of five:**" Implement

(1) destructor,
(2) copy constructor,
(3) copy assignment operator,
(4) move constructor,
(5) move assignment operator.

"**Rule of three:**"

(1) destructor,
(2) copy constructor,
(3) copy assignment operator.

Most often you will then also need to implement **(0)** a constructor.

(4) flyttingskonstruktør
(5) flyttingsoperator

At least implement **(1) the destructor**!
If **(2)** and **(3)** are not there, **forbid copying**.

Rule of five: (4) Move constructor

The **move constructor** is called when the content of an *old object* can be shifted to a *new object* that is *allocated and initialized* (*e.g.*, *before we deallocate the old object*).

```
T::T(T&& old) { … }
```

```
T func(…) {
  T tfirst;
  …
  return tfirst;
  // the destructor will be called
}

int main() {
  // but before, call the move constructor
  T tsecond = std::move( func(…) );
}
```

Typical use case: Efficient **handover of content** returned by a function.

```
class T
{
public:
  T() { this->p = new S[1000](); }

  T(T&& old) {
    this->p = old.p;
    old.p = nullptr;
  }
  …

private:
  S* p …
}
```

A **shallow copy** of the pointer to the content is good enough; after the action, the content *exists in memory only once*!

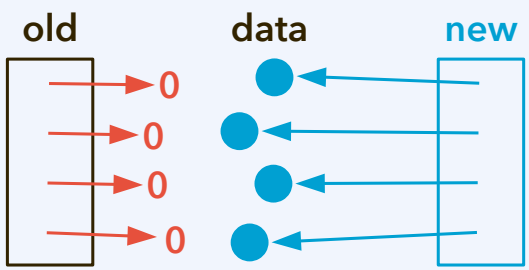
Attention: Right after the move constructor for "this", the destructor of "old" might be called.

Remove all pointers to the content from old, so that it does not get deallocated!

Move: Why can it be advantageous?

Move constructor + destructor:

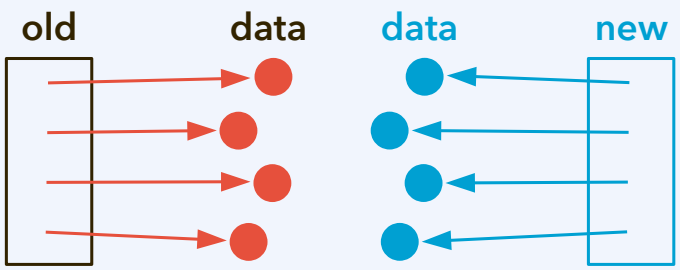
The move constructor is used to *make a new container own the data* without copying the data. A **shallow copy** is made, and the *data are detached* from the old container.



The shallow copy is an inexpensive operation. If the data exist **once in memory** both before the operation and after, *why copy them* from one place to another?

Copy constructor + destructor:

If there is *no move* constructor, or the compiler does not enforce a move, first all the content is copied (**deep copy**); the old container is probably *deallocated right after*.



This is an expensive operation whenever there is a substantial amount of data. All data are *copied*, *unnecessarily*, since at the end they still exist only **once in memory**.

Rule of five: (5) Move assignment operator

The move assignment operator relates to the move constructor the same way as the copy assignment operator relates to the copy constructor.

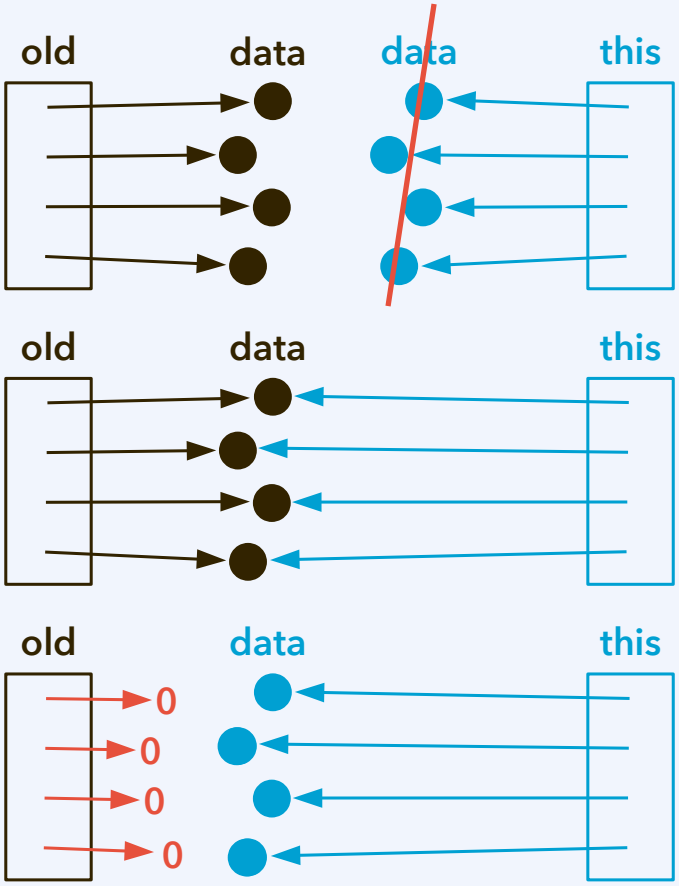
```
T& T::operator=(T&& old) { … }
```

```
T func(…) {
  T tfirst;
  …
  return tfirst;
  // the destructor will be called
}

int main() {
  T tsecond;
  …
  // but before, call the move assignment operator
  tsecond = std::move( func(…) );
}
```

constructor called

tsecond exists already



Glossar

Container (konteinar)

Definisjon: klasse/objekt som har eigarskap av minneinnhald, dvs. som utøver minnehandteringsansvaret for andre dataelement eller objekt

Flytting

Definisjon: det å overføre minneinnhaldet av éin container til ein annan

Trearregelen

Definisjon: det at det ved utvikling av kode for containerklasser som regel er behov for å implementere (1) destruktøren, (2) kopieringskonstruktøren og (3) kopieringsoperatoren

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

INF205

Ressurseffektiv programmering

2 Minnehandtering

2.1 Kallstakken og haugen

2.2 Vanlege minnehandteringsfeil

2.3 Containerklasser (kopiering og flytting)